Technical Requirements Document

Project: Autonomous ML Research Agent | **Frameworks:** Next.js, FastAPI, FastMCP | **Model:** Claude 3.5 Sonnet

1. System Architecture

The application utilizes a decoupled, three-tier architecture to separate the user interface, the AI reasoning, and the tool execution environments:

- **Frontend (Next.js):** Handles the client-side UI, state management for the chat interface, and HTTP POST requests to the backend.
- **Middleware Host (FastAPI + LangChain):** Receives queries, manages conversation history, and utilizes LangChain's `MCPAdapter` to expose server tools to the LLM.
- **Execution Server (FastMCP):** A sandboxed Python subprocess communicating via `stdio` that holds the API credentials and executes the actual Python functions.

2. Technology Stack & Dependencies

Layer	Technologies	Key Libraries
Frontend	Next.js (App Router), React 18+	Tailwind CSS, lucide-react
Backend Host	FastAPI, Uvicorn	`langchain[mcp]`, `langchain-anthropic`, `pydantic`
MCP Server	Python 3.10+	`fastmcp`, `arxiv`, `kaggle`, `pandas`

3. MCP Server Manifest & Tools

The FastMCP server exposes three strictly typed tools for the LLM to execute:

- `search_academic_papers`: Inputs: `topic` (str), `max_results` (int). Uses the `arxiv` package. Returns JSON string of titles, summaries, and PDF URLs.
- `find_kaggle_datasets`: Inputs: `keyword` (str), `max_results` (int). Uses the Kaggle REST API via local auth token. Returns JSON of dataset IDs, sizes, and ratings.
- `trigger_data_pipeline`: Inputs: `dataset_id` (str). Uses Python's `subprocess.run()` to execute the data cleaning pipeline locally.

4. Security & Compliance Protocol

- **Credential Isolation:** The `kaggle.json` token and Anthropic API keys must reside exclusively on the backend layer.
- **Subprocess Guardrails:** The `trigger_data_pipeline` tool must strictly sanitize the `dataset_id` input using a Regex pattern (alphanumerics, dashes, and one slash) to prevent command injection vulnerabilities.
- **Transport Security:** Communication between the FastAPI Host and FastMCP Server utilizes standard I/O (`stdio`), ensuring no exposed network ports on the execution layer.

5. Directory Architecture

```
/ml-research-platform
├── /frontend                  # Next.js Application
│   ├── /app/page.tsx          # Main Chat UI
│   └── /components/Chat.tsx
├── /backend                   # FastAPI Host
│   ├── main.py                # LangChain Agent & Endpoints
│   └── requirements.txt
├── /mcp_server                # Execution Layer
│   ├── ml_research_server.py  # FastMCP Tool Definitions
│   └── pipeline.py            # Pandas data-cleaning script
└── /processed_data            # Output directory for pipeline.py
```